

Эксперименты, метрики, код представлены в [ноутбуке](#).

Сначала я начала писать код нормально, декомпозируя по файлам, но сервер, на который я рассчитывала, выставил мне лимит на использование ресурсов. Пришлось делать всё на Kaggle. Это было непросто.

1. Данные

Взят датасет [google-research-datasets/mbpp](#), предназначенный для решения задач по написанию небольших функций на Python по заданному описанию, который содержит колонки

prompt – короткая инструкция по написанию функции на Python,

code – эталонный код,

test_list – тест кейсы, проверяющие решение, в среднем 3-5 штук,

test_imports (опционально) – дополнительные импорты, необходимые для тестирования написанных функций или их работоспособности

prompt	code	test_imports	test_list
string · lengths	string · lengths	list · lengths	list · lengths
			
78..116 37.5...	93..139 26.7...	0 100%	3..4 80%
Write a python function to find the first repeated character in a given string.	<pre>def first_repeated_char(str1): for index,c in enumerate(str1): if str1[:index+1].count(c) > 1: return c</pre>	[]	<pre>["assert first_repeated_char(\"abcabc\") == \"a\"", "assert first_repeated_char(\"abc\") == None", "assert first_repeated_char(\"123123\") == \"1\""]</pre>

train - 120 примеров

validation - 43 примера

2. Верификация ответов (награда за ответы)

Реализовано в `verify_solution`. Через `exec` и `eval` выполняется код. Добавлена проверка на заикливание.

Присваивается

1.0 – все тесты проходят,

0.0 – не корректен хотя бы 1 тест, выполнение функции упало с ошибкой, код написан с ошибкой, превышено время, отведенное на выполнение функции.

В качестве “To Do” для дальнейшей работы можно было бы улучшить `reward` до более плавного сигнала - награждать в соответствии с кол-вом пройденных тестов, штрафовать за заикливание и некомпилируемый код соответствующими разными штрафами. Ниже в п.5 приведены кривые обучения, они шумные, бинарная награда потенциальная причина этому.

3. Отбор Hard примеров. Baseline

В этом и последующих пунктах `model = Qwen/Qwen2.5-0.5B-Instruct`

Параметры тестирования бейзлайна

```

CONFIG = {
    "num_samples": 32, # максимальное кол-во попыток генерации, сокращено до 32 из-за ресурсов
    "temperature": 1.0,
    "top_k": 50,
    "max_new_tokens": 128, # достаточно для генерации кода небольших функций на python
    "model": "Qwen/Qwen2.5-0.5B-Instruct",
    "dtype": "float16" if torch.cuda.is_available() else "float32",
}

```

Pass@k рассмотрен по значениям: `PASS_K_VALUES = [1, 4, 8, 16, 32]`

Используется `FastLanguageModel` из **unsloth** для ускорения

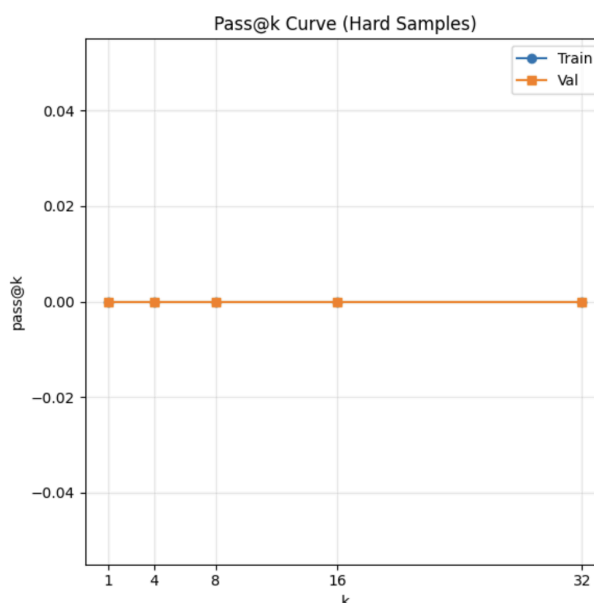
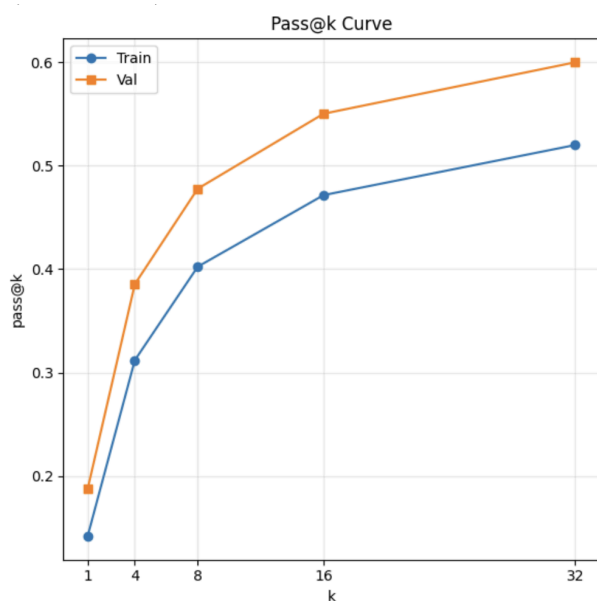
Из 120 train примеров найдено **48** hard-примеров, из 43 val примеров **12** hard. Индексы сохранены в [файле](#)

Train Hard Idx: 48/120

Val Hard Idx: 12/43

Pass@K Curve

Pass@K Curve для всех примеров (120 и 43) и Pass@k Curve для Hard-Only примеров (в подтверждение, что $\text{pass}@32 = 0$)



Средняя длина генерации в токенах (все примеры), Train: 114.47, Val: 111.02

Пример генерации кода baseline для не-hard примера

Prompt Text:

Write a python function to find the first repeated character in a given string. Give the function a name like this: first_repeated_char.

Test List:

```
['assert first_repeated_char("abcabc") == "a"', 'assert first_repeated_char("abc") == None', 'assert first_repeated_char("123123") == "1"']
```

Test Imports:

Full Messages:

```
[{'role': 'system', 'content': 'You are a helpful assistant that writes Python code.\nGiven a problem description, implement a function that solves it.\nReturn only the code. Avoid explanations, markdown formatting and test cases.'}, {'role': 'user', 'content': 'Problem: Write a python function to find the first repeated character in a given string. Give the function a name like this: first_repeated_char.\n\nImplement the function in Python:'}]
```

Generated Output:

```
```python
def first_repeated_char(string):
 char_set = set()

 for char in string:
 if char in char_set:
 return char
 else:
 char_set.add(char)
 ...
```

This implementation uses a set data structure to keep track of characters we have seen so far. It iterates over each character in the input string, and checks whether it is already in the set. If it is not, it's added to the set. If it is, then the function returns the current character as its result. This approach has a time complexity of  $O(n)$  where  $n$  is the length of the

Verification Result: True

## Пример генерации кода baseline для Hard примера

---

Prompt Text:

Write a python function to find the first repeated character in a given string. Give the function a name like this: first\_repeated\_char.

Test List:

```
['assert first_repeated_char("abcabc") == "a"', 'assert first_repeated_char("abc") == None', 'assert first_repeated_char("123123") == "1"']
```

Test Imports:

Full Messages:

```
[{'role': 'system', 'content': 'You are a helpful assistant that writes Python code.\nGiven a problem description, implement a function that solves it.\nReturn only the code. Avoid explanations, markdown formatting and test cases.'}, {'role': 'user', 'content': 'Problem: Write a python function to find the first repeated character in a given string. Give the function a name like this: first_repeated_char.\n\nImplement the function in Python:'}]
```

Generated Output:

Sure! Here's how you can implement the `first\_repeated\_char` function:

```
```python
def first_repeated_char(s):
    # Initialize an empty dictionary to keep track of characters we've seen so far
    char_seen = {}

    # Iterate through each character in the string
    for char in s:
        # Check if the character has already been seen
        if char in char_seen:
            return char # Return the first occurrence of the character

    # If the character is not in the dictionary, mark it as seen and move on to the next one
    else:
        char_seen[char] =
```

Verification Result: False

4. GRPO-Only

Модель + LoRA для более быстрого обучения

max_lora_rank=16, max_seq_length=2048

model = FastLanguageModel.get_peft_model(# сначала подгрузка модели, затем применение LoRA

model,

r = 16,

target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],

lora_alpha = 16,

use_gradient_checkpointing = "unsloth",

random_state = 33,

)

GRPO Config

```

GRPO_CONFIG = GRPOConfig(
    output_dir="./grpo_checkpoints",
    num_generations=8,
    max_completion_length=CONFIG["max_new_tokens"], # 128
    temperature=CONFIG["temperature"], # 1.0
    top_k=CONFIG["top_k"], # 50
    learning_rate=2e-5,
    beta=0.1,
    epsilon=0.2,
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,
    logging_steps=5,
    report_to="none",
    log_completions=True,
    save_steps=50,
    seed=33,
    bfl6=False,
    fp16=True,
    remove_unused_columns=False,
)

```

Модель отдает output в формате:

```

[{'role': 'assistant', 'content': ``python\ndef min_k(records, k):\n    records.sort(key=lambda x: x[1])\n\n    min_records = records[:k]\n    return min_records\n``}]

```

Сводная таблица с метриками на Val, Val-Hard представлена в конце отчета

Кривые обучения в сравнении в след пункте

Примеры генераций и Advantage

Prompt	Completion	verifier_reward	Advantage
system You are a helpful assistant that writes Python code. Given a problem description, implement a function that solves it. Return only the code. Avoid explanations, markdown formatting and test cases. user Problem: Write a function to rotate a given list by specified number of items to the right direction. https://www.geeksforgeeks.org/python-rotate-list-in-python/. Give the function a name like this: rotate_right. Implement the function in Python: assistant	<pre> python def rotate_right(arr, n, k): k = k % n # Handle case where k is greater than n k = n - k # Calculate the number of items to rotate to the left arr[n:], arr[:n] = arr[:k], arr[k:] # Rotate the array return arr </pre> You can call the `rotate_right` function by passing the list and the number of rotation items as arguments in its last parameter. For example, `rotate_right([1, 2, 3, 4], 2, 2)` will return	0.00	-0.35
system You are a helpful assistant that writes Python code. Given a problem description, implement a function that solves it. Return only the code. Avoid explanations, markdown formatting and test cases. user Problem: Write a function to divide two lists element wise. Give the function a name like this: div_list. Implement the function in Python: assistant	<pre> python def div_list(list1, list2): # divide list1 by list2, # return the elements of the resulting lists as a single list return [a/b for a, b in zip(list1, list2)] </pre>	1.00	0.94

5. SFT + GRPO

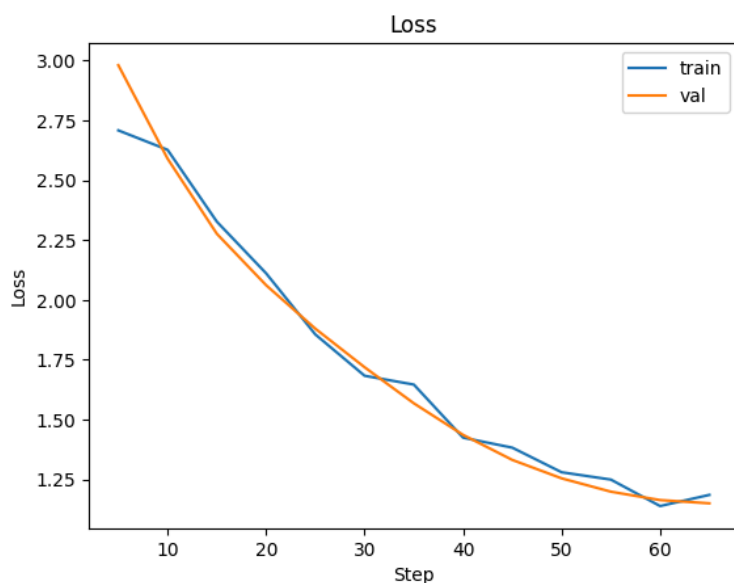
Такая же конфигурация модель + LoRA

```
sft_config = SFTConfig(  
    output_dir="/sft_checkpoints",  
    num_train_epochs=5,  
    per_device_train_batch_size=4,  
    gradient_accumulation_steps=2,  
    learning_rate=2e-5,  
    logging_steps=5,  
    save_steps=50,  
    fp16=True,  
    dataset_text_field="text",  
    eval_strategy="steps",  
    eval_steps=5,  
    packing=False,  
    max_seq_length=2048,  
    report_to="none",  
)
```

Пример сэмпла

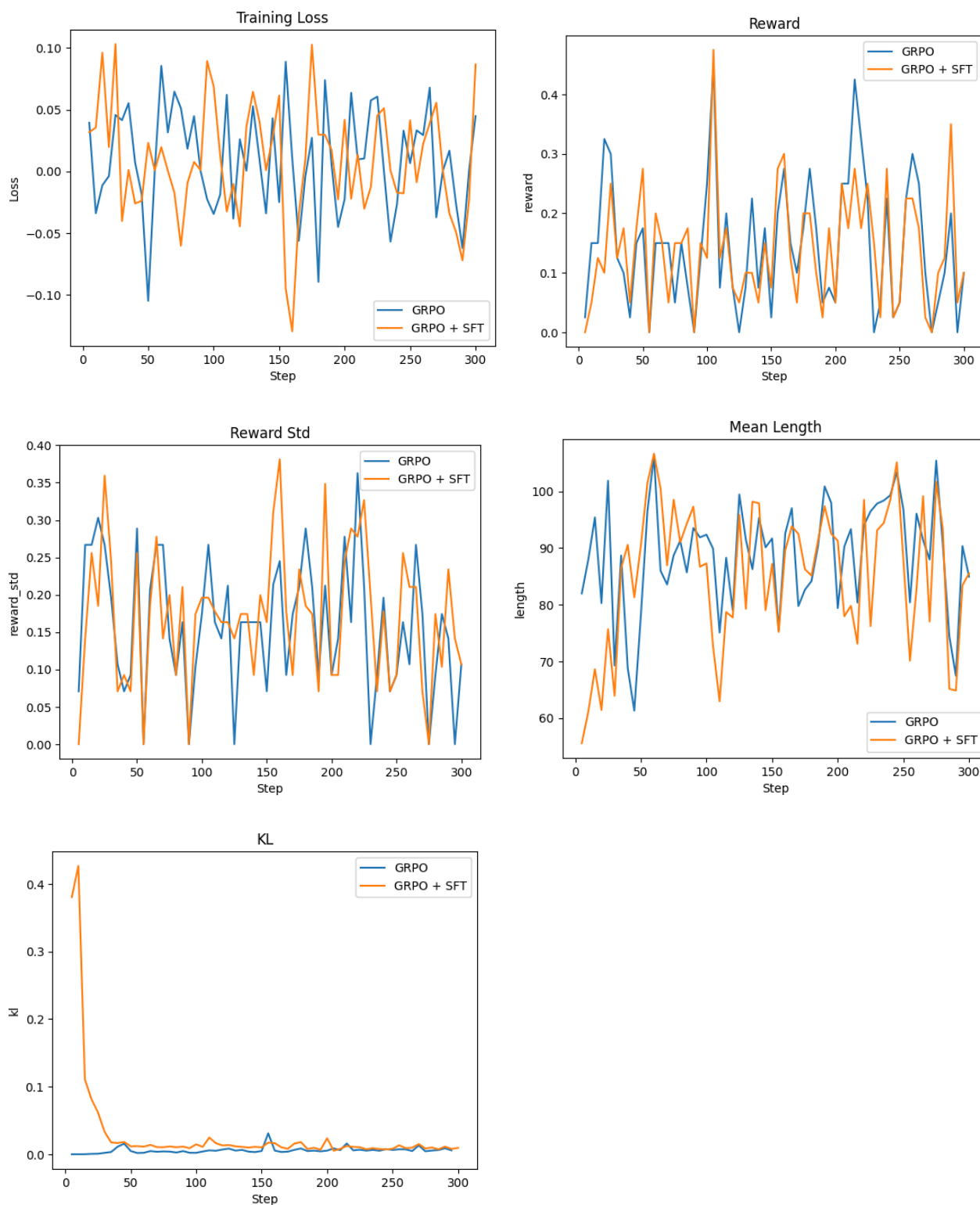
```
<|im_start|>system  
You are a helpful assistant that writes Python code.  
Given a problem description, implement a function that solves it.  
Return only the code. Avoid explanations, markdown formatting and test cases.<|im_end|>  
<|im_start|>user  
Problem: Write a python function to find the first repeated character in a given string.  
  
Implement the function in Python:<|im_end|>  
<|im_start|>assistant  
[{'role': 'assistant', 'content': 'def first_repeated_char(str1):\n    for index,c in enumerate(str1):\n        if str1[:index+1].count(c) > 1:\n            return c'}]<|im_end|>  
>
```

Лосс падает на train и val. Возможно, можно было бы попробовать большее кол-во эпох (>5), но так как train примеров всего 100, это могло бы привести к переобучению.



Последующий GRPO-config аналогичен конфигу из п.4.

Кривые обучения



Относительно loss, reward, reward std сделать выводов, скорее, не могу, так как графики шумные (высокая дисперсия). Может быть, SFT + GRPO версия достигает в reward и reward std пиков выше, чем GRPO-only. Помимо бинарной награды, все же, размер батча и датасета малы. Также причиной может быть `loga_rank=16`, который может медленнее стабилизироваться в отличие от `rank=32, 64` и т.д.

Отмечу, что на некоторых степах `reward = 0` и `reward_std = 0`, что значит, что в группе все ответы были одинаковые. Скорее всего, все одинаково плохие). Поле для проб - можно попробовать DAPO с фильтрацией на такие вещи или LUFFY (попытки реализовать который были..).

На графике средней длины можно увидеть, что GRPO+SFT версия сначала выдает генерации ~60 токенов, но затем выравнивается с GRPO-only. Это влияние SFT версии, что будет ниже показано в сводных таблицах, что длина генерации там на ~30% меньше.

KL-дивергенция у GRPO + SFT сначала очень большая, относительно GRPO-only, что можно также заметить по появлению неадекватных генераций на первых степах обучения:

<pre>system You are a helpful assistant that writes Python code. Given a problem description, implement a function that solves it. Return only the code. Avoid explanations, markdown formatting and test cases. user Problem: Write a python function to find the smallest missing number from a sorted list of natural numbers. Give the function a name like this: find_First_Missing. Implement the function in Python: assistant</pre>	<pre>from sortedcontainers import SortedList def find_First_Missing(a): -</pre>	0.00	0.00
<pre>system You are a helpful assistant that writes Python code. Given a problem description, implement a function that solves it. Return only the code. Avoid explanations, markdown formatting and test cases. user Problem: Write a python function to find the smallest missing number from a sorted list of natural numbers. Give the function a name like this: find_First_Missing. Implement the function in Python: assistant</pre>	<pre>[-2, 0, 1, 3, 5]</pre>	0.00	0.00
<pre>system You are a helpful assistant that writes Python code. Given a problem description, implement a function that solves it. Return only the code. Avoid explanations, markdown formatting and test cases. user Problem: Write a python function to find the smallest missing number from a sorted list of natural numbers. Give the function a name like this: find_First_Missing. Implement the function in Python: assistant</pre>	<pre>[{'role': 'assistant', 'title': 'function to find the first missing natural number in a sorted list.', 'arguments': {}, 'answer': 'import sys\n'.format(sys)}]</pre>	0.00	0.00

На последующих степах генерации нормализируются и начинают принимать вид, похожий как в GRPO-only логам.

6. Сводная Pass@k Val по экспериментам

Exp	pass@1	pass@4	pass@8	pass@16	pass@32	avg len (tokens)
Baseline	0.151	0.3136	0.41	0.483	0.53	96
GRPO	0.2146	0.38	0.461	0.523	0.57	95
SFT	0.064	0.183	0.268	0.366	0.467	52
SFT + GRPO	0.217	0.367	0.4475	0.5266	0.6	84

Pass@k Hard Val по экспериментам

Exp	pass@1	pass@4	pass@8	pass@16	pass@32	avg len (tokens)
Baseline	0	0	0	0	0	104
GRPO	0.01	0.039	0.07	0.12	0.167	102
SFT	0.0052	0.02	0.042	0.083	0.167	61
SFT + GRPO	0.0182	0.054	0.075	0.083	0.083	95

pass@32 = 0 удалось сделать > 0.

Baseline имеет самую большую среднюю длину. Без обучения под конкретную задачу модель может выдавать излишние объяснения, пытаться рассуждать. После SFT длина сокращается практически на треть, так как в target codes в датасете приведены только коды функций, без комментариев и docstring. После применения GRPO после SFT длина становится прежней (на стадии RL модель снова исследует пространство), также после GRPO-only длина не изменяется почти. Может быть додумано, но SFT + GRPO выглядит как компромисс между лаконичностью и верными решениями.

Применение SFT сильно снижает метрики, хуже, чем бейзлайн. Скорее всего, это из-за того что модель запомнила правильные примеры и потеряла способность к импровизации. Энтропию не удалось залогировать, но, думаю, там она снизилась, что плохо для поиска новых решений на OOD данны. На hard-примерах SFT+GRPO является наилучшей комбинацией для pass@1-8. Применение GRPO и в Only оказывает положительное влияние.

GRPO лучше по метрикам на больших k, SFT + GRPO на меньших. Можно предположить, что это из за более высокой энтропии при GRPO, следовательно, больше разнообразие при сэмплировании и лучше при большом k.

Тоже как путь для улучшения - все таки взять больше данных и повторить эксперименты на большем кол-ве hard примеров.